

Project Title:

Real-Time Scalable Fraud Detection System Using Hybrid AI and Rule-Based Models

Duration – 2 Months

Objective:

To build a **real-time fraud detection system** that analyzes **large-scale financial transactions** using a combination of **supervised ML**, **unsupervised anomaly detection**, and **rule-based logic**, with full audit trails and compliance with financial regulations such as **AML** and **GDPR**. The system should support **multi-channel integrations** (banking apps, payment gateways, mobile wallets), process **millions of events per day**, and provide **analyst-level explainability**.

Functional Requirements:

1. User Authentication and Authorization

- Role-based access for Admin, Fraud Analyst, Auditor, Data Scientist, and Customer Support.
- MFA and OAuth2-based SSO integration.
- Fine-grained RBAC policies for data viewing, case handling, and audit log access.

2. Advanced Transaction Monitoring

- Stream processing of real-time transactions using Apache Kafka or AWS Kinesis.
- Assign risk categories dynamically using a scoring engine.
- Time-based anomaly patterns (e.g., suspicious activity bursts in short intervals).

3. Hybrid Fraud Detection Engine

- **Supervised Learning Models:** Train classifiers (XGBoost, LightGBM, Random Forest) on labeled transaction data.
- **Unsupervised Learning:** Use autoencoders, Isolation Forest, or One-Class SVM to detect unseen anomalies.
- Ensemble models for final fraud prediction with confidence intervals.

4. Dynamic Rule Engine

- Analysts can define, test, and deploy detection rules (e.g., >3 withdrawals in 2 min).
- Historical performance score for each rule: False Positives/Negatives over time.

- Supports fuzzy logic and logical chaining of rules.

5. Alerting and Notification System

- Real-time push alerts to analysts when a transaction crosses the risk threshold.
- Escalation path for high-severity alerts (SMS, Slack, email, dashboard).
- Alert fatigue detection (notify on novel fraud behavior only).

6. Investigation and Case Lifecycle

- One-click conversion of flagged transactions into investigation cases.
- Assign to fraud analysts, attach evidence, notes, model output, and transaction history.
- Integration with internal chat for collaboration on ongoing cases.

7. Behavioral and Contextual Analytics

- Track individual customer behavior over time for baseline profiling.
- Device fingerprinting and geo-behavior analysis (sudden location changes, device switch).
- Time-series clustering to group abnormal sessions.

8. Customizable Dashboards and Reporting

- Dashboards for executive summary, high-risk users, rule performance, model drift.
- Time-series trend graphs, risk score heatmaps, and fraud funnel visualizations.
- PDF/CSV/Excel/JSON export support and auto-scheduled reporting.

9. Predictive and Prescriptive Analytics

- Use time-series forecasting to predict future fraud spikes.
- Prescriptive suggestions: flag accounts for KYC revalidation or freeze accounts.
- Feature importance dashboards for model interpretability.

10. Integrations and APIs

- REST APIs and Webhooks for integrations with banking systems, AML tools, and customer identity verification systems.
 - Plug-and-play adapters for Stripe, Razorpay, Plaid, UPI systems.
 - ETL pipelines for data ingestion from legacy systems and cloud stores.
-

Non-Functional Requirements:

1. Performance

- Handle >10 million transactions per day with 1–2 second latency for fraud decisions.
- Low memory footprint for model inference and rule execution.

2. Scalability

- Microservices architecture supporting horizontal scaling.
- Scalable ML pipelines using Spark MLlib or AWS SageMaker.

3. Availability and Reliability

- 99.9% uptime with active-passive failover.
- Graceful degradation: switch to rule-only fallback if ML service is down.

4. Security

- Full encryption (AES-256 for storage, TLS 1.3 for transport).
- Anti-replay, IP whitelisting, and anomaly-based DDoS detection.
- Secure logging with tamper-proof audit logs.

5. Compliance and Auditing

- GDPR-compliant user data access and deletion modules.
- AML compliance: transaction trails, KYC record syncing, suspicious activity reports (SAR).
- Full audit logs of model versions used for each prediction.

6. Backup & Disaster Recovery

- Daily and hourly backups of transaction logs and models.
- Multi-region disaster recovery plan with RTO < 1 hour.

7. Model Governance & Explainability

- Track versions of all ML models deployed.
- Provide explainable AI (SHAP, LIME) for analyst understanding of fraud scores.
- Model performance metrics (AUC, precision-recall, drift) on live data.

8. Cross-Platform Access

- Responsive dashboard for mobile analysts (React or Flutter-based).

- iOS/Android support for viewing alerts and case actions.

9. Documentation and Training

- Internal wiki, OpenAPI 3.0 documentation for APIs.
- Internal LMS module with case studies on fraud detection techniques.
- Onboarding simulator for new analysts to test fraud classification.

Technical Stack:

Component	Technology Stack
Backend	Python (FastAPI), Node.js
ML/AI	Scikit-learn, TensorFlow, PyTorch, XGBoost
Stream Processing	Apache Kafka, Apache Flink / AWS Kinesis
Data Storage	PostgreSQL, MongoDB, Amazon S3, ElasticSearch
Dashboard & UI	React.js + TailwindCSS
Authentication	OAuth2.0, JWT, Keycloak
DevOps & Deployment	Docker, Kubernetes, GitHub Actions, Helm
Monitoring & Logging	Prometheus, Grafana, ELK Stack
APIs	RESTful + OpenAPI (Swagger)
Hosting	AWS / Azure / GCP

Implementation Timeline (2 Months):

Week Milestone

- 1 Project setup, user auth, stream ingestion setup with Kafka
- 2 Implement rule engine + build basic dashboards
- 3 Train ML models (supervised + unsupervised), define risk score logic
- 4 Integrate real-time transaction scoring (ML + rules)

Week Milestone

- 5 Alerts module, fraud case lifecycle setup
- 6 Behavioral profiling, dashboard expansion, SHAP explainability
- 7 Reporting engine, PDF/CSV exports, API documentation
- 8 End-to-end testing, security hardening, model audit logs, deployment